

Instituto Politécnico Nacional

Escuela Superior de Cómputo

Web Client and Backend Development Frameworks.

Investigación: los códigos de respuesta del protocolo HTTP

Profesor: Dr. José Asunción Enríquez Zárate

Alumno: Angel Frausto Robles

Boleta: 2019630177

id634296@gmail.com

7CM2

19 de septiembre de 2026

Índice

1. Introducción	2
2. Qué es HTTP	2
3. Para qué sirve el código de respuesta	2
4. Las cinco familias	3
5. Qué código devolver en cada operación	5
6. Los errores de elección más frecuentes	5
7. Qué cambia con HTTPS	6
8. Conclusión	6
9. Referencias Bibliográficas	7

Índice de figuras

1. El código de estado es la única parte de la respuesta que los intermediarios pueden interpretar . 3

Índice de tablas

1.	Las cinco familias de códigos de respuesta y su significado	3
2.	Códigos informativos, de éxito y de redirección	4
3.	Códigos de error del cliente y del servidor	4
4.	Códigos que corresponden a cada operación sobre el recurso carreras	5

1. Introducción

Toda respuesta HTTP empieza con un número de tres dígitos. Ese número es lo primero que lee el cliente y, en muchos casos, lo único que necesita leer: dice si la petición salió bien, si el recurso se movió, si quien preguntó se equivocó o si el problema ocurrió del lado del servidor. El cuerpo de la respuesta viene después y a veces ni siquiera existe.

Este documento trata sobre esos números. Empieza por lo que es HTTP y por qué su diseño obliga a que cada respuesta se explique sola. Sigue con las cinco familias de códigos y con lo que significa cada una. Después baja al detalle de cuáles corresponden a cada operación en una interfaz de tipo REST y cuáles se eligen mal con frecuencia. Cierra con lo que agrega la S de HTTPS y, sobre todo, con lo que no cambia.

El ejemplo que recorre el texto es una interfaz sobre un recurso llamado **carreras**, con las operaciones de crear, consultar, actualizar y eliminar. Sirve para tener un caso concreto donde cada código aparezca en la situación que le toca.

Queda fuera el detalle del transporte: cómo se negocia una conexión TLS, cómo HTTP/2 multiplexa varios flujos sobre una sola conexión y cómo HTTP/3 los traslada a QUIC. Esos temas aparecen solo donde hacen falta para entender el contexto. El asunto del documento es la semántica de la respuesta, que es idéntica en las tres versiones del protocolo.

2. Qué es HTTP

HTTP es un protocolo de capa de aplicación para transferir representaciones de recursos entre un cliente y un servidor [1]. Funciona por turnos: el cliente manda una petición, el servidor devuelve una respuesta, y ahí termina el intercambio.

La característica que más consecuencias tiene es que el protocolo no guarda estado. Cada petición llega sin memoria de las anteriores, y el servidor no está obligado a recordar nada entre una y otra. Todo lo que el servidor necesita para atender una petición tiene que venir dentro de esa misma petición. Las sesiones de usuario, los carritos de compra y las banderas de «ya inició sesión» existen porque alguien las construyó encima, con cookies o con tokens, no porque HTTP las provea.

Una petición tiene un método, un destino, la versión del protocolo, una lista de encabezados y un cuerpo opcional. Una respuesta tiene una línea de estado, encabezados y un cuerpo también opcional. La línea de estado es donde vive el código:

Código 1: Un intercambio completo, con la línea de estado en la primera línea de la respuesta

```
1 GET /api/carreras/7 HTTP/1.1
2 Host: localhost:8080
3 Accept: application/json
4
5 HTTP/1.1 200 OK
6 Content-Type: application/json
7 Content-Length: 96
8
9 {"idCarrera":7,"nombre":"Ingenieria en Sistemas Computacionales","clave":"ISC"}
```

El 200 es el código. El OK que lo acompaña se llama frase de razón y está pensado para que una persona lea el intercambio, no para que el programa decida con ella. RFC 9110 es explícito en esto: un cliente no debe tomar decisiones a partir de la frase, porque puede venir en cualquier idioma o venir vacía [1].

Ese detalle deja de ser teórico en cuanto se cambia de versión. HTTP/1.1 manda todo eso como texto, tal como se ve arriba. HTTP/2 y HTTP/3 comprimen los encabezados en formato binario y eliminaron la frase de razón por completo: en esas versiones lo único que viaja es el número. Un cliente que hubiera hecho depender su lógica del texto «OK» deja de funcionar al actualizar el protocolo, mientras que uno que solo mira el 200 sigue igual.

3. Para qué sirve el código de respuesta

El código responde a una pregunta muy concreta: qué debe hacer ahora quien recibió la respuesta. Y la responde en un formato que sirve tanto para el programa que hizo la petición como para todo lo que hay en medio.

Eso último es lo que suele pasarse por alto. Entre el cliente y el servidor casi nunca hay un cable directo. Hay cachés, proxies inversos, balanceadores de carga y redes de distribución de contenido, y ninguno de ellos entiende de carreras ni de categorías. Lo único que pueden interpretar es el código, y con él toman decisiones reales: una caché guarda por su cuenta una respuesta 200 para reutilizarla, y no guarda una 503 porque sabe

que describe una falla pasajera. Un balanceador que recibe varias respuestas 502 seguidas de la misma máquina puede sacarla de rotación.

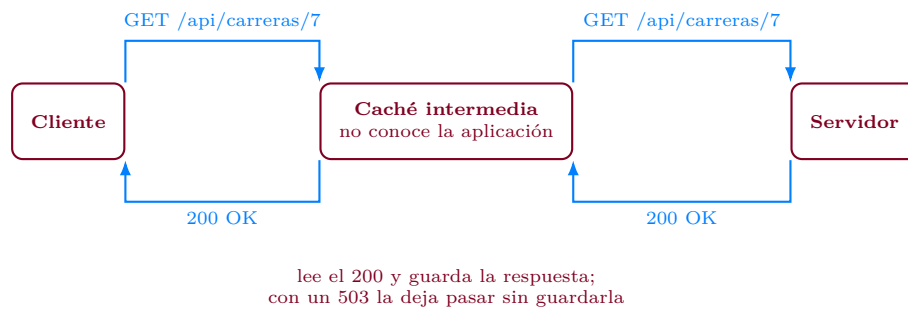


Figura 1: El código de estado es la única parte de la respuesta que los intermediarios pueden interpretar

La figura 1 explica por qué el código es un número y no una descripción libre. Un texto sirve para que alguien lo lea; un número de un conjunto acotado sirve para que una máquina que no sabe nada del dominio actúe correctamente.

Hay una segunda propiedad del diseño que conviene conocer. El primer dígito, por sí solo, es un contrato. RFC 9110 obliga a que un cliente que recibe un código desconocido lo trate como el código `x00` de su familia [1]. Si mañana aparece un 452 que nadie ha visto, el cliente correcto lo maneja como un 400 genérico y sigue funcionando. Por eso el registro oficial de códigos que mantiene la IANA puede crecer sin romper a nadie [2].

4. Las cinco familias

Los códigos van del 100 al 599 y se agrupan por su primer dígito. La tabla 1 resume qué afirma cada grupo y qué se espera que haga el cliente al recibirlo.

Familia	Nombre	Qué afirma	Qué hace el cliente
1xx	Informativa	La petición se recibió y el proceso continúa	Esperar; no es la respuesta final
2xx	Éxito	La petición se recibió, se entendió y se atendió	Seguir adelante
3xx	Redirección	Hace falta un paso más para completarla	Ir a donde indique el encabezado
4xx	Error del cliente	La petición está mal formada o no procede	Corregir y volver a intentar
5xx	Error del servidor	La petición parecía válida pero el servidor falló	Reintentar más tarde o avisar

Tabla 1: Las cinco familias de códigos de respuesta y su significado

La frontera entre 4xx y 5xx es la más importante de las cuatro, porque reparte la culpa. Un 4xx dice que reintentar la misma petición tal cual va a fallar otra vez. Un 5xx dice que la petición estaba bien y que reintentarla más tarde tiene sentido. Confundirlas hace que los clientes reintenten lo que nunca va a funcionar o que se rindan ante algo que se habría resuelto solo.

Los códigos que aparecen en la práctica son pocos. Las tablas 2 y 3 los reúnen uno por uno; después de cada tabla se explica el detalle que no cabe en una descripción de una línea.

Código	Nombre	Descripción
100	Continue	Indica al cliente que puede enviar el cuerpo que anunció.
101	Switching Protocols	Confirma el cambio de protocolo, como al abrir un WebSocket.
200	OK	Respuesta de éxito genérica.
201	Created	Indica que se creó un recurso nuevo; suele venir con el encabezado Location .
202	Accepted	El trabajo se encoló y todavía no termina.
204	No Content	La operación salió bien y a propósito no hay cuerpo que devolver.
301	Moved Permanently	El recurso cambió de dirección para siempre.
302	Found	Desvío temporal; a diferencia de 307, el cliente puede cambiar el método al seguirlo.
307	Temporary Redirect	Desvío temporal que obliga a conservar el método original.
304	Not Modified	Respuesta a una petición condicional cuando el recurso no cambió; viaja sin cuerpo.

Tabla 2: Códigos informativos, de éxito y de redirección

Código	Nombre	Descripción
400	Bad Request	Error de cliente genérico; se usa para datos que no pasan validación.
401	Unauthorized	Falta de credenciales; debe venir con el encabezado WWW-Authenticate .
403	Forbidden	Credenciales válidas pero sin permiso suficiente para la operación.
404	Not Found	El servidor no pudo encontrar el contenido solicitado.
405	Method Not Allowed	La ruta existe pero no acepta el método usado.
409	Conflict	Choque con el estado actual, como una clave que ya está ocupada.
429	Too Many Requests	Se excedió un límite de peticiones permitidas.
500	Internal Server Error	Error de servidor genérico; suele ser una excepción sin atender.
502	Bad Gateway	Un intermediario recibió una respuesta inválida del servicio de atrás.
503	Service Unavailable	El servicio está temporalmente fuera de servicio.
504	Gateway Timeout	Un intermediario no recibió respuesta a tiempo del servicio de atrás.

Tabla 3: Códigos de error del cliente y del servidor

100 Continue le dice al cliente que puede mandar el cuerpo que anunció. **101 Switching Protocols** es la respuesta a un cambio de protocolo, y es la que se ve al abrir un WebSocket. En una interfaz REST común no se escriben a mano.

200 OK es la respuesta con éxito genérica. **201 Created** indica que se creó un recurso, y lo habitual es acompañarlo del encabezado **Location** con la dirección del recurso nuevo. **202 Accepted** sirve cuando el trabajo se encoló y todavía no termina. **204 No Content** significa que todo salió bien y que a propósito no hay cuerpo que devolver.

301 Moved Permanently avisa que la dirección cambió para siempre y que conviene actualizar el enlace. **302** y **307** indican un desvío temporal, con la diferencia de que 307 obliga a conservar el método original. **304 Not Modified** es la respuesta a una petición condicional cuando el recurso no ha cambiado, y viaja sin cuerpo porque la copia que tiene el cliente sigue siendo válida.

400 Bad Request es el error de cliente genérico y el que se usa para datos que no pasan validación. **401 Unauthorized** indica falta de credenciales. **403 Forbidden** indica credenciales válidas sin permiso suficiente. **404 Not Found** dice que el recurso no existe. **405 Method Not Allowed** dice que la ruta existe pero no acepta ese método. **409 Conflict** corresponde a un choque con el estado actual, como intentar dar de alta algo cuya clave ya está ocupada. **429 Too Many Requests** indica que se excedió un límite de peticiones.

500 Internal Server Error es el error de servidor genérico y suele significar que una excepción llegó hasta arriba sin que nadie la atendiera. **502 Bad Gateway** y **504 Gateway Timeout** los devuelve un intermediario

cuando el servicio de atrás respondió mal o no respondió. **503 Service Unavailable** indica indisponibilidad temporal.

5. Qué código devolver en cada operación

Con las cinco operaciones sobre el recurso **carreras** se cubren casi todas las situaciones que se presentan. La tabla 4 las reúne.

Petición	Éxito	Errores esperables
POST /api/carreras	201 Created	400 si no valida, 409 si la clave ya existe
GET /api/carreras	200 OK	500 si algo falla del lado del servidor
GET /api/carreras/7	200 OK	404 si no existe esa carrera
PUT /api/carreras/7	200 OK o 204	400 si no valida, 404 si no existe
DELETE /api/carreras/7	204 No Content	404 si no existe, 409 si tiene dependencias

Tabla 4: Códigos que corresponden a cada operación sobre el recurso **carreras**

Dos casos de esa tabla merecen explicación.

El alta devuelve 201 y no 200 porque el código dice algo que el cuerpo no dice: que existe un recurso nuevo. Conviene que la respuesta traiga además el encabezado **Location** apuntando a él, para que el cliente sepa dónde quedó sin tener que armar la ruta por su cuenta:

Código 2: Respuesta al alta de una carrera

```
1 HTTP/1.1 201 Created
2 Location: /api/carreras/12
3 Content-Type: application/json
4
5 {"idCarrera":12,"nombre":"Ingenieria en Inteligencia Artificial","clave":"IIA"}
```

El listado vacío es 200 y no 404. La colección **/api/carreras** existe siempre, aunque hoy no tenga elementos, y la respuesta correcta es un 200 con un arreglo vacío. El 404 queda reservado para cuando se pide un elemento concreto que no está. Es una distinción sutil que rompe a los clientes cuando se decide mal: quien recibe un 404 al pedir el listado no puede saber si la base está vacía o si escribió mal la ruta.

6. Los errores de elección más frecuentes

Elegir mal el código no suele romper nada de inmediato, y por eso los errores sobreviven mucho tiempo. Estos cuatro son los que más se repiten.

El primero es devolver 200 con un cuerpo que dice que hubo un error, algo del estilo `{.exit": false, "mensaje": "no se encontro"}`. Para todo lo que hay entre el cliente y el servidor esa respuesta es un éxito, así que una caché la puede guardar y servirla después a alguien más. El cliente queda obligado a leer y entender el cuerpo para saber qué pasó, que es justo lo que el código de estado existía para evitar.

Tratar 401 y 403 como sinónimos es el segundo. Un 401 significa que el servidor no sabe quién está preguntando, y RFC 9110 exige que venga acompañado del encabezado **WWW-Authenticate** indicando cómo autenticarse [1]. Un 403 significa lo contrario: el servidor sí sabe quién es y aun así no lo deja pasar. Mandar 403 a un usuario sin sesión lo deja sin enterarse de que lo que le falta es iniciarla.

El tercero aparece cuando un error de validación termina saliendo como 500. El 500 afirma que el servidor falló, y eso desvía la atención hacia donde no está el problema. También manda la señal equivocada al cliente, que va a reintentar una petición condenada a fallar otra vez. Un campo obligatorio que llegó vacío es un 400.

Queda el de contestar 404 con la ruta correcta y el método equivocado. Si **/api/carreras/7** responde a GET pero no a PATCH, la respuesta a un PATCH es 405, con el encabezado **Allow** listando los métodos que sí se aceptan. Con un 404 el cliente concluye que el recurso no existe, cuando el problema era otro.

Conviene mencionar una excepción, porque parece un error y no lo es. Algunos servicios devuelven 404 en lugar de 403 a propósito, para no revelar que un recurso existe a quien no tiene permiso de verlo. Es una decisión de seguridad deliberada, y la diferencia con el descuido está en que se toma a conciencia y se documenta.

7. Qué cambia con HTTPS

HTTPS no es un protocolo distinto. Es el mismo HTTP transportado sobre una conexión TLS, y RFC 9110 lo define exactamente así al describir el esquema de URI `https` [1]. Los métodos son los mismos, los encabezados son los mismos y los códigos de respuesta son los mismos. Un 404 sobre HTTPS es el mismo 404.

Lo que cambia está debajo. El puerto por omisión pasa de 80 a 443. Antes del primer byte de HTTP se realiza un saludo TLS en el que el servidor presenta un certificado firmado por una autoridad en la que el cliente confía, y las dos partes acuerdan las llaves con las que van a cifrar el resto de la conversación [3]. A partir de ahí, cualquiera que observe la red ve tráfico cifrado en vez de texto: no puede leer las rutas, ni los encabezados, ni los cuerpos, ni las cookies, y tampoco puede modificarlos sin que se note.

Lo que no cambia importa igual. TLS protege el tramo entre el cliente y el servidor, y nada más. No autentica al usuario, que sigue necesitando su propio mecanismo de sesión. No valida lo que llega, así que un cuerpo malicioso viaja cifrado con la misma comodidad que uno legítimo. No oculta con quién se está hablando, porque la dirección IP del servidor es visible y el nombre del sitio viaja en claro dentro del saludo TLS salvo que se active una extensión adicional que lo cifre. Y no dice nada sobre lo que ocurre una vez que los datos llegaron: si el servidor guarda contraseñas sin cifrar, el certificado no lo remedia.

8. Conclusión

El código de estado va separado del cuerpo a propósito. Quien tiene que entenderlo no siempre entiende la aplicación: el cliente decide con él si continuar o reintentar, y una caché decide si guardar la respuesta o dejarla pasar de largo. Ninguno de los dos necesita abrir el JSON para eso.

Por eso escogerlo mal cuesta más de lo que parece. Un 200 encima de un error convierte una falla en algo que las cachés van a repartir. Un 500 encima de un dato mal escrito manda a buscar el problema al lugar equivocado, y de paso invita al cliente a reintentar una petición que nunca va a pasar. Un 403 donde iba un 401 deja al usuario sin enterarse de que lo único que le faltaba era iniciar sesión. Y un 404 donde iba un 405 le hace creer que el recurso no existe. En los cuatro casos el sistema sigue de pie, que es justamente lo que permite que el error sobreviva años.

Para las dudas del día a día basta con acertarle al primer dígito, que es el que reparte la responsabilidad entre el cliente y el servidor. El segundo y el tercero afinan el motivo, y para eso conviene tener a la mano el registro de la IANA [2].

Angel Frausto Robles

9. Referencias Bibliográficas

- [1] Roy T. Fielding, Mark Nottingham y Julian Reschke. *HTTP Semantics*. Inf. téc. RFC 9110. Internet Engineering Task Force, 2022. URL: <https://www.rfc-editor.org/rfc/rfc9110.html> (visitado 07-09-2026).
- [2] Internet Assigned Numbers Authority. *Hypertext Transfer Protocol (HTTP) Status Code Registry*. URL: <https://www.iana.org/assignments/http-status-codes/http-status-codes.xhtml> (visitado 07-09-2026).
- [3] Eric Rescorla. *The Transport Layer Security (TLS) Protocol Version 1.3*. Inf. téc. RFC 8446. Internet Engineering Task Force, 2018. URL: <https://www.rfc-editor.org/rfc/rfc8446.html> (visitado 07-09-2026).